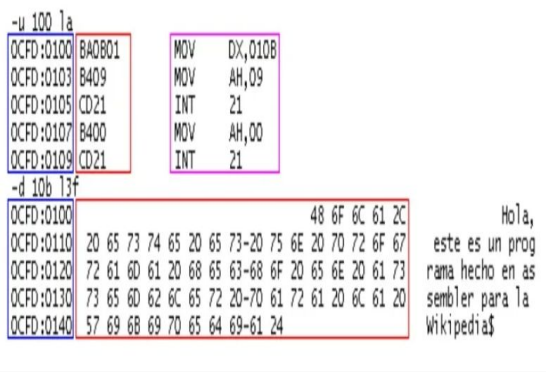
Tema 6

Programación de bases de datos

Introducción. Conceptos generales

En esta unidad se va estudiar cómo incorporar la funcionalidad que aporta un lenguaje de programación de tercera generación con elementos como definición de variables, sentencias condicionales, bucles, etc; e integrarlo a las distintas sentencias que aporta SQL.

Introducción. Conceptos generales



1º Generación



2º Generación



3º Generación

Introducción. Conceptos generales

1º Generación

- Nivel más bajo, directamente entendible por el hardware
- Código binario
- Específico para cada arquitectura de computadora



2º Generación



3º Generación

Introducción. Conceptos generales

1º Generación

- Nivel más bajo, directamente entendible por el hardware.
- Código binario.
- Específico para cada arquitectura de computadora.

2º Generación

- Usa mnemónicos en lugar de código binario.
- Más legible para humanos pero aún de bajo nivel.
- Requiere un ensamblador para traducir a máquina.



3º Generación

Introducción. Conceptos generales

1º Generación

- Nivel más bajo, directamente entendible por el hardware.
- Código binario.
- Específico para cada arquitectura de computadora.

2º Generación

- Usa mnemónicos en lugar de código binario.
- Más legible para humanos pero aún de bajo nivel.
- Requiere un ensamblador para traducir a máquina.

3º Generación

- Abstracción del hardware.
- Sintaxis más cercana al lenguaje humano.
- Portables entre diferentes sistemas.
- Incluyen estructuras de control, tipos de datos, etc.

Introducción. Lenguaje de Programación

Durante los temas anteriores nos hemos dedicado a aprender a utilizar el lenguaje **SQL (Lenguaje de consulta estructurado)**, el cual como ya sabemos permite crear, acceder y modificar datos almacenados en bases de datos relacionales.

Con SQL podemos interactuar con la información almacenada en la BD a través de consultas como por ejemplo:

```
select nombre, Edad, Posición
from jugadores
where ID_equipo =
(select ID_equipo
from equipos
where Nombre = 'Sevilla FC') and Edad > 30;
```

	nombre	Edad	Posición
▶	Tomas Vaclik	32	Portero
	Aleix Vidal	32	Defensa
	Jesús Navas	35	Defensa
	Fernando	33	Centrocampista
	Ivan Rakitić	33	Centrocampista
	Papu Gómez	33	Centrocampista
	Franco Vázquez	32	Centrocampista

Introducción. Lenguaje de Programación

La sentencia vista antes es realmente simple, ya que la acción que pretende realizar sobre la BD también lo es. Simplemente era realizar una consulta sobre los jugadores de un determinado equipo filtrando aquellos que su edad no supera los 30 años.

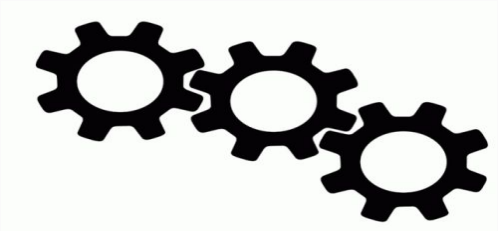
```
select nombre, Edad, Posición
from jugadores
where ID_equipo =
(select ID_equipo
from equipos
where Nombre = 'Sevilla FC') and Edad > 30;
```

	nombre	Edad	Posición
▶	Tomas Vaclik	32	Portero
	Aleix Vidal	32	Defensa
	Jesús Navas	35	Defensa
	Fernando	33	Centrocampista
	Ivan Rakitić	33	Centrocampista
	Papu Gómez	33	Centrocampista
	Franco Vázquez	32	Centrocampista

Introducción. Lenguaje de Programación

Pero supongamos que queremos mantener actualizada la edad de los jugadores en la base de datos, aumentándole automáticamente cada vez que llega su cumpleaños.

Para lograr esto, necesitaríamos escribir un procedimiento que se ejecute periódicamente y que consulte la fecha del sistema. Luego, el procedimiento compararía la fecha de nacimiento de cada jugador con la fecha del sistema y, si corresponde, aumentaría la edad del jugador en la base de datos.



Introducción. Lenguaje de Programación

Para realizar este tipo de tareas, necesitamos definir variables, estructuras condicionales estructuras de control, definición de funciones, etc.

Aquí entrarían los lenguajes de programación procedural que nos permitían integrar todas estas funcionalidades con el propio lenguaje SQL



Introducción. Lenguaje de Programación

PL/SQL es un **lenguaje procedimental** estructurado en bloques que amplía la funcionalidad de SQL.

Con PL/SQL podemos usar sentencias SQL para manipular datos y sentencias de control de flujo para procesar los datos. Por tanto, **PL/SQL combina la potencia de SQL** para la manipulación de datos, con la potencia de los lenguajes procedimentales para procesar los datos.

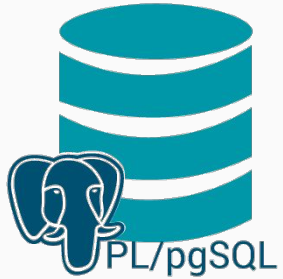
Aunque **PL/SQL fue creado por Oracle**, hoy día todos los gestores de bases de datos utilizan **un lenguaje procedimental muy parecido** al ideado por Oracle para poder programar las bases de datos.



Introducción. Lenguaje de Programación

- **Oracle Database:** Utiliza **PL/SQL (Procedural Language/Structured Query Language)** como su lenguaje de programación procedural. PL/SQL se utiliza para escribir **procedimientos almacenados, funciones, desencadenadores y otros bloques de código que se ejecutan dentro de la base de datos**.
- **PostgreSQL:** Utiliza un lenguaje procedural llamado **PL/pgSQL (Procedural Language/PostgreSQL Structured Query Language)** para escribir **procedimientos almacenados, funciones y desencadenadores.** PL/pgSQL es específico de PostgreSQL y proporciona capacidades similares a PL/SQL de Oracle.





Introducción. Lenguaje de Programación

- **MySQL:** MySQL tiene su propio sistema de procedimientos almacenados que se basa en SQL y **no tiene un lenguaje procedural específico** como PL/SQL o PL/pgSQL**.** En MySQL, puedes escribir **procedimientos almacenados, funciones y desencadenadores directamente utilizando SQL estándar**. Sin embargo, **MySQL admite algunas extensiones de SQL para procedimientos almacenados, conocidas como SQL/PSM (SQL Persistent Stored Modules)**, que proporcionan algunas características adicionales para la escritura de código procedural.



Procedimientos Almacenados

El código más básico en **PL/SQL** son los **bloques anónimos** que se pueden introducir y ejecutar directamente en la consola SQL.

Los bloques anónimos son **secuencias de instrucciones o comandos** que se ejecutan de forma independiente en el entorno de la base de datos **sin ser nombrados o almacenados de manera permanente**.

Dado que **MySQL no admite bloques anónimos**, la alternativa es **crear procedimientos almacenados para lograr funcionalidades similares.**

Triggers, procedimientos y funciones en MySQL

Junto con otras cosas, lo principal que veremos durante la unidad son:

- **Procedimiento almacenado:** Es un objeto que se **crea con la sentencia CREATE PROCEDURE** y **se invoca con la sentencia CALL**. **Un procedimiento puede tener 0 o muchos parámetros de entrada** y **cero o muchos parámetros de salida**.
- **Función almacenada:** Es un objeto que se **crea con la sentencia CREATE FUNCTION** y **se invoca con la sentencia SELECT** o **dentro de una expresión**. **Una función puede tener 0 o muchos parámetros de entrada** y **siempre devuelve un valor**, asociado al nombre de la función.
- **Trigger:** Es un objeto que se **crea con la sentencia CREATE TRIGGER** y tiene que estar **asociado a una tabla**. **Se activa cuando ocurre un evento de inserción, actualización o borrado**, sobre la tabla a la que está asociado.

Procedimientos Almacenados

Un procedimiento almacenado en MySQL es un conjunto de instrucciones SQL que se almacenan en el servidor de la base de datos y se pueden invocar y ejecutar repetidamente.

Esto también existe en Oracle y recibe el mismo nombre, la principal diferencia es que MySQL nos priva de la posibilidad de crear los bloques anónimos.

Procedimientos Almacenados

```
DELIMITER //
CREATE PROCEDURE saludo()
BEGIN
    SELECT '¡Hola, mundo!' AS mensaje;
END //
DELIMITER ;
```

```
CALL saludo();
```

	mensaje
▶	¡Hola, mundo!

Procedimientos Almacenados. DELIMITER

Para definir un procedimiento almacenado es necesario modificar temporalmente el **carácter separador** que se utiliza para delimitar las sentencias SQL.

Cuando definimos **funciones, procedimientos almacenados y triggers** en los que debe **definir varias instrucciones SQL**. Utilizamos la sentencia **DELIMITER** para definir temporalmente un **nuevo separador,** este se utiliza para **definir el final de todo el procedimiento,** ya dentro del mismo irán varias sentencias SQL que utilizan **;** como separador de las mismas. De esta manera definimos el procedimiento como un bloque completo.

Procedimientos Almacenados. DELIMITER

Podemos utilizar el delimitador que nosotros queramos intentando siempre buscar caracteres o conjunto de caracteres que no puedan hacer conflictos con otras instrucciones.



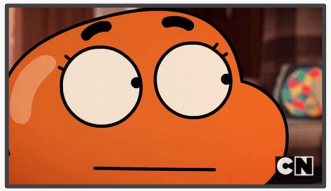
```
DELIMITER //
```

```
DELIMITER $$
```

```
DELIMITER |||
```

```
DELIMITER GAMUSINO
```

```
DELIMITER supercalifragilisticoespialidoso
```





```
DELIMITER $
```

```
DELIMITER @
```

```
DELIMITER %
```

Procedimientos Almacenados. BEGIN…END

La estructura **BEGIN** … **END** se utiliza para agrupar un conjunto de instrucciones. Si un **procedimiento** o una **función** necesita contener más de una instrucción en su interior, éstas deberán aparecer dentro de un BEGIN … END.

Hacer esto nos permite poder agrupar las instrucciones en bloques de ejecución pudiendo anidar bloques y estructurar nuestro código.

Procedimientos Almacenados. Parámetros

En los procedimientos almacenados podemos tener **3 tipos de parámetros:**

- **Entrada:** Se indican poniendo la palabra reservada **IN delante del nombre del parámetro.** Estos parámetros **no pueden cambiar su valor dentro del procedimiento**. En programación sería equivalente al paso por valor de un parámetro.
- **Salida:** Se indican poniendo la palabra reservada **OUT delante del nombre del parámetro**. Estos parámetros **cambian su valor dentro del procedimiento**. En programación sería equivalente al paso por referencia de un parámetro.
- **Entrada/Salida:** Es una combinación de los tipos **IN y OUT**. Estos parámetros se indican poniendo la palabra reservada **IN/OUT delante del nombre del parámetro**.

Ejercicio

- Vamos a usar **Jardinería.** Crea un procedimiento llamado lista_de_productos que reciba un parámetro de entrada que sea gama de un producto y muestre mediante un select aquellos productos que coincidan con la gama proporcionada por el parámetro.

```
call lista_de_productos('Ornamentales');
```

Ejercicio

- Crea un procedimiento llamado cuenta_productos que reciba como entrada el nombre de la gama y devuelva el número de productos que existen de esa gama.

Para asignar valor a una variable se utiliza la sentencia SET:

SET *Nombre_variable* =

MySQL permite utilizar variables para almacenar resultados y utilizarlas después. Las variables se identifican porque van precedidas por un carácter ‘@’.

Ejercicio

- Crea un procedimiento que se llame MaxMinMed, que reciba como parámetro de entrada el nombre de la gama de un producto y devuelva como salida tres parámetros. El precio máximo, el precio mínimo y la media de los productos que existen en esa gama.

Variables

Mientras hemos hecho procedimientos hemos estado utilizando algunas variables, pero en ningún momento las hemos declarado.

Eso se debe a que en MySQL, cuando utilizamos variables dentro de un procedimiento, no es necesario declararlas explícitamente antes de su uso. Ya que cuando ponemos parámetros de entrada o de salida, MySQL, crea estas variables para su uso dentro del proceso.

Pero…¿Si queremos utilizar una variable que no es ni de entrada ni de salida?

Variables

Declaración de variables

```
DELIMITER //
CREATE PROCEDURE Calculadora_Notas2
(IN Nota_Actividad_1 int, IN Nota_Actividad_2 int, IN Nota_Actividad_3 int, IN Nota_Actividad_4 int, IN Nota_Examen_1 int, IN Nota_Examen_2 int, OUT Nota int)
BEGIN
declare media_Actividades decimal(4,2);
declare media_Examenes decimal(4,2);
SET media_Actividades = ((Nota_Actividad_1 + Nota_Actividad_2 + Nota_Actividad_3 + Nota_Actividad_4)/4);
SET media_Examenes = ((Nota_Examen_1 + Nota_Examen_2)/2);
SET Nota = media_Actividades *0.45 + media_Examenes*0.55;
END //
```

Variables

Cuando trabajamos con variables, es importante entender la diferencia entre las v**ariables locales** y las **variables de usuario**.

- **Variables locales:** Estas son las variables que declaras dentro del procedimiento almacenado utilizando la palabra clave **DECLARE**. Existen solo dentro del procedimiento.
- **Variables de usuario:** Son aquellas que puedes declarar y usar fuera de los procedimientos. Para distinguirlas, se usa el símbolo @ cuando las declaras y las utilizas.

Funciones

Tanto las funciones como los procedimientos almacenados son objetos que puedes crear y utilizar para realizar tareas específicas en una base de datos.

En el caso de la funciones para crearlas usaremos **CREATE FUNCTION,** seguido del nombre que le queramos poner y entre paréntesis los parámetros de entrada.



Puesto que las funciones devuelven un parámetro de salida y esta devolución se realiza con la sentencia **RETURN.** Dejamos de necesitar especificar si los parámetros que definimos serán de entrada o de salida(**Ya no será necesario poner el IN o el OUT).**

Funciones

En la definición de la cabecera de la función hay que definir el tipo de dato que se devolverá usando la palabra reservada **RETURNS** y en el cuerpo de la función debemos incluir la palabra reservada **RETURN** para devolver el valor.

```
DELIMITER $$
CREATE FUNCTION Función_Ejemplo( x VARCHAR(50))
 RETURNS INT
BEGIN
 RETURN Resultado;
END
$$
```

Funciones

Después de utilizar el RETURNS, tenemos que indicar las características de la función. Las opciones disponibles son las siguientes:

- **DETERMINISTIC:** Indica que la función siempre devuelve el mismo resultado cuando se utilizan los mismos parámetros de entrada.
- **NOT DETERMINISTIC:** Indica que la función no siempre devuelve el mismo resultado, aunque se utilicen los mismos parámetros de entrada.

Funciones

- **CONTAINS SQL:** Indica que la función contiene sentencias SQL, pero no contiene sentencias de **manipulación de datos**, es decir, no se va a actualizar, insertar o borrar ningún dato.
- **NO SQL:** Indica que la función no contiene sentencias SQL.
- **READS SQL DATA:** Indica que la función no modifica los datos de la base de datos y que contiene sentencias de lectura de datos.
- **MODIFIES SQL DATA:** Indica que la función sí modifica los datos de la base de datos y que actualizará, insertará o borrará algún dato.

Funciones

Para poder crear una función en MySQL es necesario indicar al menos una de estas tres características:

- **DETERMINISTIC**
- **NO SQL**
- **READS SQL DATA**

Si no se indica al menos una de estas características obtendremos un mensaje de error.

Funciones

Es posible configurar el valor de la variable global, para indicar a MySQL que queremos eliminar la restricción de indicar alguna de las características anteriores cuando definimos una función. Esta variable está configurada con el valor 0 por defecto y para poder modificarla es necesario contar con el privilegio SUPER. La forma de modificarla es:

```
SET GLOBAL log_bin_trust_function_creators = 1;
```

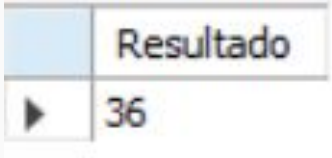
Ejercicio

- Crea una función llamada calcular_area_circulo que calcule el área de un círculo dado su radio. Utilizalo la función en una consulta select simple.

	Area
▶	1324.12

Ejercicio

- Crea una función llamada sumar_serie que reciba un número entero como parámetro y devuelva la suma de todos los números enteros desde 1 hasta el número dado.



	Resultado
▶	36

Ejercicio

- Crea una función llamada calcular_total_gasto_cliente que reciba el código de un cliente como parámetro y devuelva el total de gasto realizado por ese cliente.

	total_ventas_cliente
▶	10365.00

Funciones vs Procedimientos

Tanto las funciones como los procedimientos almacenados son objetos que puedes crear y utilizar para realizar tareas específicas. Sin embargo, hay diferencias entre ellos:

- **Valor de retorno.**
- **Lugar de uso.**

Funciones vs Procedimientos

Valor de retorno

- **Funciones:** Pueden devolver un valor, como un **entero**, una **cadena de caracteres** o un **booleano**, utilizando la cláusula **RETURN**. Esto significa que **puedes usar el resultado de una función en una expresión SQL**, como parte de una consulta.
- **Procedimientos almacenados:** No devuelven valores directamente. **Pueden realizar operaciones y manipulaciones en la base de datos**, pero **no pueden ser utilizados en una expresión SQL** directamente para obtener un valor.

Funciones vs Procedimientos

Lugar de Uso

- **Funciones:** Pueden ser **utilizadas en cualquier lugar** donde se permita una expresión, como en una consulta **SELECT**, en una cláusula **WHERE**, etc.
- **Procedimientos almacenados:** Puedes llamar a un procedimiento almacenado desde **una aplicación o desde otra parte del código SQL**, pero **no puedes utilizarlos directamente dentro de una consulta**.

Estructuras de control

Las estructuras de control al igual que en cualquier otro lenguaje de programación, son herramientas que nos permiten controlar cómo se ejecuta un programa, tomar decisiones basadas en ciertas condiciones y/o repetir acciones según sea necesario.

Es decir, como programadores nos permiten tener control sobre el flujo de nuestro programa.



Estructuras de control

Las estructuras de control que vamos a ver en esta unidad son:

- **Instrucciones condicionales.**
  - **IF-THEN-ELSE**
  - **CASE**
- **Instrucciones repetitivas o bucles.**
  - **LOOP**
  - **REPEAT**
  - **WHILE**



IF-THEN-ELSE

De igual manera que habéis visto en Java, las instrucciones condicionales permiten ejecutar un bloque de código dependiendo de si la condición evaluada es verdadera o falsa(TRUE o FALSE, incluso 0 o 1 también sería equivalente). Podemos dividir para mayor facilidad la sentencia IF en dos tipos:

- **Condicional IF Simple**
- **Condicional IF Compuesta**

IF-THEN-ELSE

```
DELIMITER //
CREATE FUNCTION Aprobado_Suspendido
(A1 INT, A2 INT, A3 INT, A4 INT, E1 INT, E2 INT)
RETURNS VARCHAR(10)
BEGIN
    DECLARE Nota decimal(10,2);
    DECLARE Estado VARCHAR(10) default 'Suspenso';
    CALL Calculadora_Notas(A1, A2, A3, A4, E1, E2, Nota);

    IF Nota >= 5 THEN
        SET Estado = 'Aprobado';
    END IF;
    RETURN Estado;
END //

DELIMITER ;
```

```
CREATE FUNCTION Aprobado_Suspendido2
(A1 INT, A2 INT, A3 INT, A4 INT, E1 INT, E2 INT)
RETURNS VARCHAR(30)
BEGIN
    DECLARE Nota DECIMAL(10,2);
    DECLARE Estado VARCHAR(30);
    CALL Calculadora_Notas(A1, A2, A3, A4, E1, E2, Nota);
    IF Nota = 9 or Nota = 10 THEN
        SET Estado = 'Sobresaliente';
    ELSEIF Nota >= 7 THEN
        SET Estado = 'Notable';
    ELSEIF Nota >= 5 THEN
        SET Estado = 'Suficiente';
    ELSEIF Nota <= 5 AND Nota >= 0 THEN
        SET Estado = 'Insuficiente';
    ELSE
        SET Estado = 'Fallo en el Calculo de notas';
    END IF;
    RETURN Estado;
END //
DELIMITER ;
```

Actividad con IF

- Crea un procedimiento almacenado llamado par_impar que tome un parámetro de entrada número de tipo INT y determine si el número es par o impar. Imprime 'Par' si el número es par y 'Impar' si es impar utilizando SELECT.
- Crea una función llamada es_bisiesto que tome un parámetro de entrada anio de tipo INT y determine si el año es bisiesto o no.

CASE

Realmente la sentencia **CASE** podría decirse que es una forma “bonita” de realizar un if que contiene varias **ELSEIF**, ya que esta nos permite realizar múltiples evaluaciones y ejecutar diferentes bloques de código según el resultado de estas evaluaciones.

```
CREATE FUNCTION dia_semana(numero INT)
RETURNS VARCHAR(10)
BEGIN
    DECLARE dia VARCHAR(10);
    CASE numero
        WHEN 1 THEN SET dia = 'Lunes';
        WHEN 2 THEN SET dia = 'Martes';
        WHEN 3 THEN SET dia = 'Miércoles';
        WHEN 4 THEN SET dia = 'Jueves';
        WHEN 5 THEN SET dia = 'Viernes';
        WHEN 6 THEN SET dia = 'Sábado';
        WHEN 7 THEN SET dia = 'Domingo';
        ELSE SET dia = 'Número inválido';
    END CASE;
    RETURN dia;
END //

DELIMITER ;
```

CASE Simple

Esta forma funciona de forma muy similar al **switch** de Java. Se utiliza cuando queremos comparar una única columna o expresión contra una lista de valores específicos.

```
CREATE FUNCTION dia_semana(numero INT)
RETURNS VARCHAR(10)
BEGIN
    DECLARE dia VARCHAR(10);
    CASE numero
        WHEN 1 THEN SET dia = 'Lunes';
        WHEN 2 THEN SET dia = 'Martes';
        WHEN 3 THEN SET dia = 'Miércoles';
        WHEN 4 THEN SET dia = 'Jueves';
        WHEN 5 THEN SET dia = 'Viernes';
        WHEN 6 THEN SET dia = 'Sábado';
        WHEN 7 THEN SET dia = 'Domingo';
        ELSE SET dia = 'Número inválido';
    END CASE;
    RETURN dia;
END //

DELIMITER ;
```

CASE con Condiciones

Esta es la versión concreta y la que más usaréis en el mundo real. No se limita a comparar un valor fijo, sino que permite usar operadores lógicos en cada rama del CASE.

```
CASE
    WHEN nota_media >= 9 THEN 'Sobresaliente'
    WHEN nota_media >= 7 THEN 'Notable'
    WHEN nota_media >= 5 THEN 'Aprobado'
    ELSE 'Suspenso'
```

Actividad con CASE

- Crea una función llamada tipo_triangulo que tome tres parámetros de entrada lado1, lado2 y lado3, todos de tipo INT, y devuelva el tipo de triángulo que forman.
- Crea un procedimiento llamado obtener_color que tome como entrada 3 enteros que representen el modelo RGB y deberá determinar el nombre del color(Blanco, Rojo, Verde, Azul, Gris u otro) basándose en los valores proporcionados y devolver el resultado en forma de cadena de caracteres.

LOOP

Las estructuras de control de repetición o bucles, son herramientas fundamentales en todo lenguaje de programación, puesto que permiten **ejecutar un conjunto de instrucciones de forma repetida** mientras se cumpla **una condición específica**.

Estas estructuras son esenciales para **automatizar tareas** que requieren realizar una misma operación múltiples veces o para p**rocesar datos de manera iterativa**.

LOOP

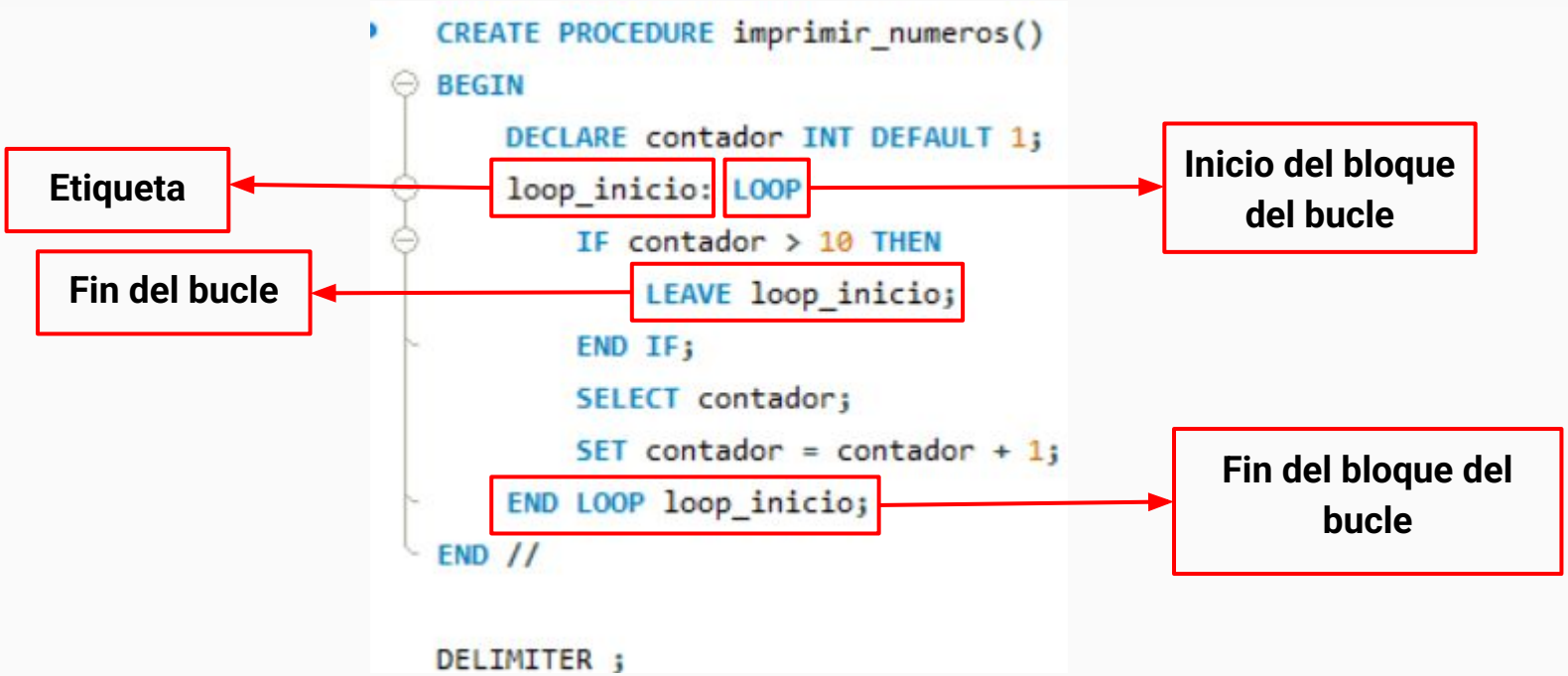
Como su mismo nombre indica(“Bucle”), es una sentencia que crea un proceso que se repetirá constantemente lo que haya en su interior hasta que “algo” lo detenga(**¡Cuidado, no vayas a crear un bucle infinito!**). El bloque de código que contiene el **LOOP** comienza con la misma palabra y su cuerpo alcanza hasta la sentencia **END LOOP**, que indica el fin del bloque(**Que no se refiere al fin del proceso de repetición**). La forma de parar las iteraciones es mediante la sentencia **LEAVE,** en el instante que esta sentencia es ejecutada, el bucle se interrumpe.

LOOP

Una buena práctica a la hora de trabajar con bucles en MySQL es agregarles etiquetas en su creación, una etiqueta es simplemente un identificador que se utiliza para marcar un punto específico en el código.

Estas son útiles cuando necesitas salir de un bucle anidado o cuando quieres especificar claramente a qué bucle pertenece una instrucción de salida.

LOOP


CREATE PROCEDURE imprimir_numeros()
BEGIN
DECLARE contador INT DEFAULT 1;
loop_inicio: LOOP
IF contador > 10 THEN
LEAVE loop_inicio;
END IF;
SELECT contador;
SET contador = contador + 1;
END LOOP loop_inicio;
END //
DELIMITER ;
Etiqueta
Fin del bucle
Inicio del bloque del bucle
Fin del bloque del bucle

REPEAT

REPEAT es una estructura de control de flujo que permite ejecutar un bloque de código repetidamente hasta que una condición específica se cumpla, hasta aquí exactamente igual que **LOOP.**

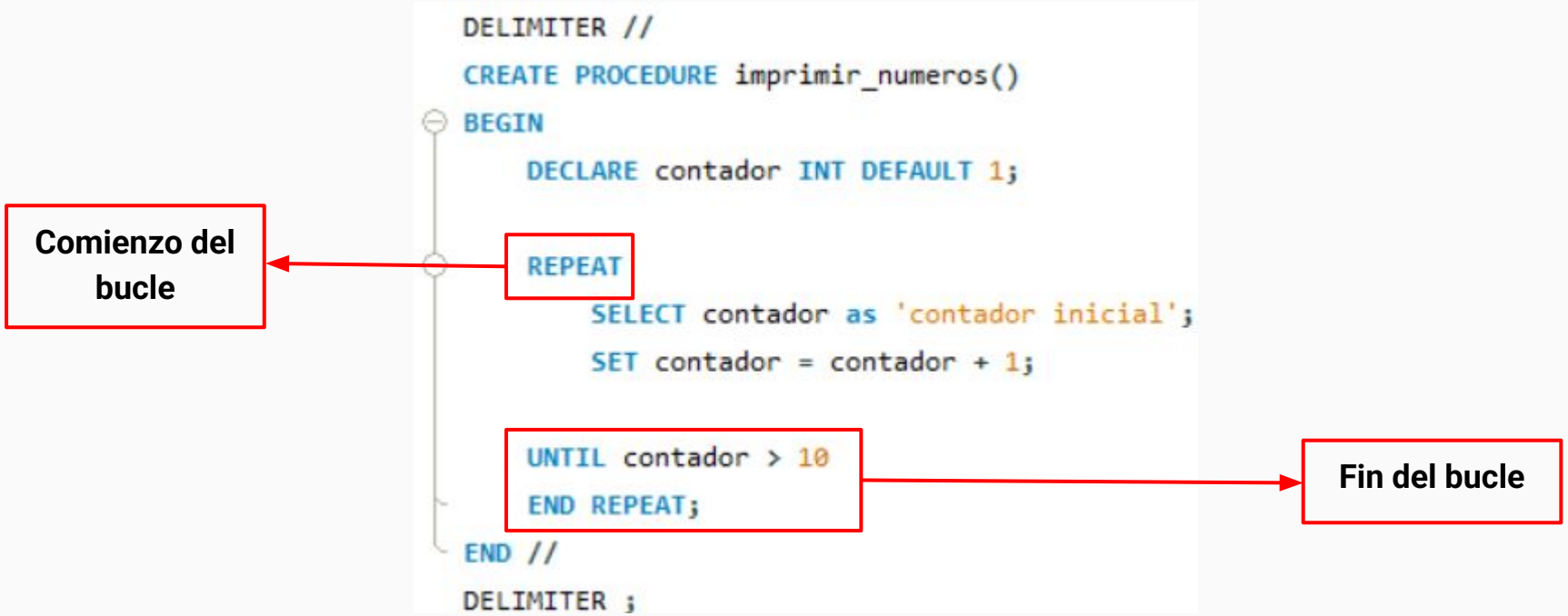
La principal diferencia que existe con LOOP es que REPEAT, al contrario que LOOP, **evalúa la condición de salida del bloque al final del mismo**, **asegurando** que las instrucciones contenidas en él, **se ejecutarán al menos una vez**.

REPEAT

Para inicializar el bloque a repetir, al igual que hemos estado haciendo hasta ahora, utilizamos la palabra reservada **REPEAT** y para indicar el final de dicho bloque utilizaremos la palabra **UNTIL junto con la condición** que utilizaremos para concluir el conjunto de instrucciones del bucle.



REPEAT


DELIMITER //
CREATE PROCEDURE imprimir_numeros()
BEGIN
DECLARE contador INT DEFAULT 1;
REPEAT
SELECT contador as 'contador inicial';
SET contador = contador + 1;
UNTIL contador > 10
END REPEAT;
END //
DELIMITER ;
Comienzo del bucle
Fin del bucle

WHILE

El bucle creado con **While** permite repetir el conjunto de instrucciones existentes dentro de su bloque mientras la condición que el mismo comprueba sea verdadera, en el momento en el que dicha condición deja de serlo, el bucle no continuará.

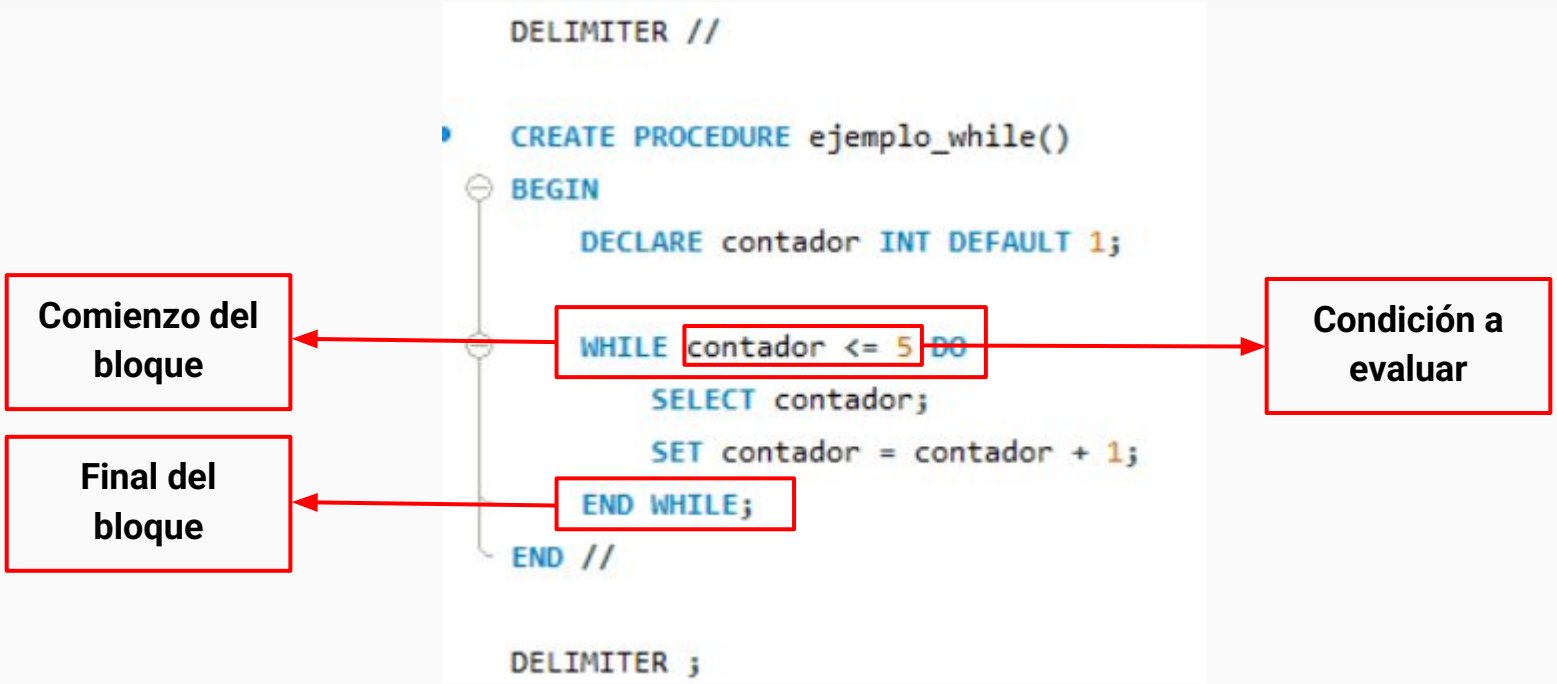
Puntualizar que en caso de que **inicialmente la condición sea falsa, no se ejecutará en ningún momento el bucle**, pero **una vez comenzado** a ejecutarse dicho bloque, **no se volverá a comprobar la condición hasta su próxima iteración.**

WHILE

Esta estructura de control da comienzo su bloque con la sentencia **WHILE *Condición* DO** y el final de dicho bloque, como viene siendo costumbre es **END WHILE; .**

Como hemos dicho antes, el **WHILE *Condición* DO** indica donde comienza, pero no necesariamente se ejecutará(Dependerá de si la condición es cierta o no) y el **END WHILE;** marca donde el bloque finaliza y comienza a repetirse de nuevo, es decir, llegar a esa sentencia no asegura salir del bucle.

WHILE


DELIMITER //
CREATE PROCEDURE ejemplo_while()
BEGIN
DECLARE contador INT DEFAULT 1;
WHILE contador <= 5 DO
SELECT contador;
SET contador = contador + 1;
END WHILE;
END //
DELIMITER ;
Comienzo del bloque
Final del bloque
Condición a evaluar

LOOP vs REPEAT vs WHILE

DIFERENCIAS			
	LOOP	REPEAT	WHILE
Condición	Se puede evaluar en cualquier parte del bloque	Se evalúa al final del bloque, antes de la siguiente ejecución	Se evalúa al comienzo del bloque, antes de comenzar la 1º ejecución
Ejecución	Permite salir del bucle en cualquier momento, dejando instrucciones sin ejecutar dentro	Permite ejecutar siempre la 1º iteración de bucle	Permite no ejecutar nada del bloque en caso de no cumplirse la condición
Sintaxis	[Etiqueta:] LOOP [Instrucciones] [condición con IF] LEAVE [Instrucciones] END LOOP [Etiqueta];	REPEAT [Instrucciones] UNTIL condición END REPEAT;	WHILE condición DO [Instrucciones] END WHILE;

Manejo de errores

Durante todos los temas anteriores hemos ido viendo viendo que con relativa facilidad podemos encontrarnos errores que detuvieran la ejecución de nuestro Script, muchos de esos errores resultar imposibles de evitar en ciertas ocasiones y solamente podemos preveer que es posible que ocurrirán en alguna ocasión.

Para llevar el control de estos errores podemos definir **handlers** en nuestro procedimientos y funciones

Manejo de errores

Si nos hemos ido fijando en nuestro **OUTPUT** cada error que se produce al ejecutar alguna instrucción genera un código concreto. Para poder manejar los errores será necesario conocer los códigos de los errores que deseamos “vigilar” para que nuestro **handler** actúe cuando estos se produzcan.



HANDLER

Un **Handler** es una estructura que te permite manejar y controlar errores que pueden ocurrir durante la ejecución de un procedimiento almacenado, una función, un trigger u otros bloques de código SQL. Es decir, nos permite anticipar estos posibles errores y proporcionar un plan de acción para manejarlos.

Cuando se utiliza un **Handler**, se especifica una condición que activará el manejo de errores, junto con un bloque de código que se ejecutará cuando esa condición se cumpla.

HANDLER

Por ejemplo, imagina que creas un procedimiento almacenado para dividir dos números. Para manejar la posibilidad de que el **divisor sea cero (lo que resultaría en un error de división por cero)**, puedes usar un **Handler** para detectar esa situación y tomar medidas para manejarla adecuadamente, cómo devolver un mensaje de error personalizado.



HANDLER

La sintaxis de un **HANDLER** es la siguiente:

```
DECLARE *handler_action* HANDLER FOR *condition_value*
    BEGIN
        # Instrucciones para manejar el error
    END;
```

HANDLER

- ***handler_action*:** Es lo que nos permitirá determinar cómo se comportará mi programa tras manejar el error que activa el handler. Hay dos posibles opciones en MySQL:
  - **CONTINUE:** Tras ejecutar el código del manejador, mi programa continuará con su normal ejecución.
  - **EXIT:** Tras ejecutar el código del manejador, el programa se detendrá.

```
DECLARE *handler_action* HANDLER FOR *condition_value*
```

HANDLER

- ***condition_value*:** Indica la condición específica o clase de condiciones que activa el handler. Puede adoptar las siguientes formas:
  - **Códigos de estado SQL (SQLSTATE):** Estos códigos son códigos numéricos o alfanuméricos que representan diferentes tipos de errores o condiciones en MySQL.
  - **Número de error de MySQL:** Es posible indicar directamente el código de error que genera MySQL por su OUTPUT.
  - **Clases de condiciones:** MySQL también permite agrupar varios códigos de estado SQL en clases de condiciones más amplias.

```
DECLARE *handler_action* HANDLER FOR *condition_value*
```

Ejemplos

Código error concreto

- 1062 : (Duplicate entry)
- 1451: (Cannot delete or update a parent row: a foreign key constraint fails)
- 1051 : (Unknown table)
- 1054: (Unknown column)

SQLSTATE

- 23000 - Violación de restricción de clave única.
- 42000 - Error de sintaxis o error de consulta.
- 28000 - Error de acceso denegado.
- 42S02 - Tabla o vista no encontrada en la base de datos.

Tabla de errores MySQL

Clases de condiciones

- SQLWARNING : Equivalente a todos los valores de SQLSTATE que empiezan con 01.
- NOT FOUND : Equivalente a todos los valores de SQLSTATE que empiezan con 02.
- SQLEXCEPTION: Equivalente a todos los valores de SQLSTATE que empiezan por 00, 01 y 02.

CURSOR

Los cursores son un objeto que declaramos, generalmente, dentro de un proceso para usarlo en un momento preciso y que nos vuelva un determinado valor.

Son estructuras de control que permiten recorrer filas de un conjunto de resultados obtenidos por una consulta SQL, una fila a la vez.

Hasta ahora las consultas cuando las ejecutamos, de forma instantánea nos devolvieron todos y cada uno de los resultados a la vez, pero…

¿Y si queremos trabajar con esos resultados de manera individual y realizar operaciones específicas con cada uno de ellos?

CURSOR

Imagina que necesitamos crear un procedimiento almacenado que genere un informe detallado de los pedidos de un cliente específico.

En lugar de obtener todos los pedidos del cliente de una vez, queremos procesar cada pedido individualmente para incluir información adicional, como los productos en cada pedido y el total gastado en cada uno.

Los cursores es la manera que tenemos de crear un bucle que recorra nuestras tablas y de esa manera poder manipular de forma individual los datos obtenidos.

CURSOR

Para implementar un cursor debemos tener en cuenta 4 fases de su funcionamiento:

- Declaración.
- Apertura.
- Lectura.
- Cierre.

CURSOR. Declaración

Al igual que una variable, los cursores se declaran con la sentencia **DECLARE**.

DECLARE *nombre_cursor* **CURSOR FOR** *consulta*;

Debemos declarar el cursor después de la declaración de variables, de lo contrario MySQL, generará un error. Cuando declaramos un cursor dentro de un **procedimiento**, tras declarar el cursor debemos declarar un manejador de errores (**HANDLER**).

CURSOR. Declaración

Cuando se está recorriendo un cursor y no quedan filas por recorrer se lanza el error **NOT FOUND**, que se corresponde con el valor **SQLSTATE '02000'**.

Por eso cuando estemos trabajando con cursores será necesario declarar un handler para manejar este error.

La sintaxis quedaría similar a esto:

CURSOR. Declaración

CREATE PROCEDURE ejemplo()

BEGIN

DECLARE *variables;

...

DECLARE cur **CURSOR FOR** *SELECT ...*;

DECLARE CONTINUE HANDLER FOR NOT FOUND ...;

CURSOR. Apertura

En la fase de declaración **la consulta a la que hace referencia el cursor, aun no se ha ejecutado**. Para ejecutarla usaremos el comando **OPEN**. Sin esta apertura los resultados del cursor no pueden ser leídos por MySQL.

Cuando abrimos el cursor esté sitúa un puntero a la primera fila devuelta por la consulta. La sentencia de apertura se vería así:

OPEN cur ;

CURSOR. Lectura

La lectura de los resultados de un cursor se hace con el comando **FETCH**. Este nos permite acceder a la primer fila generada por la consulta. **Si se vuelve a usar el cursor pasa a apuntar a la segunda fila, luego a la tercer y así sucesivamente hasta que el cursor no tenga resultados que referenciar.** Se vería así:

FETCH cur **INTO** variable1,variable2,...;

Como podemos observar lo que estamos haciendo es decir que guarde en las diferentes variables los datos que extraemos de la consulta del cursor.

CURSOR. Cierre

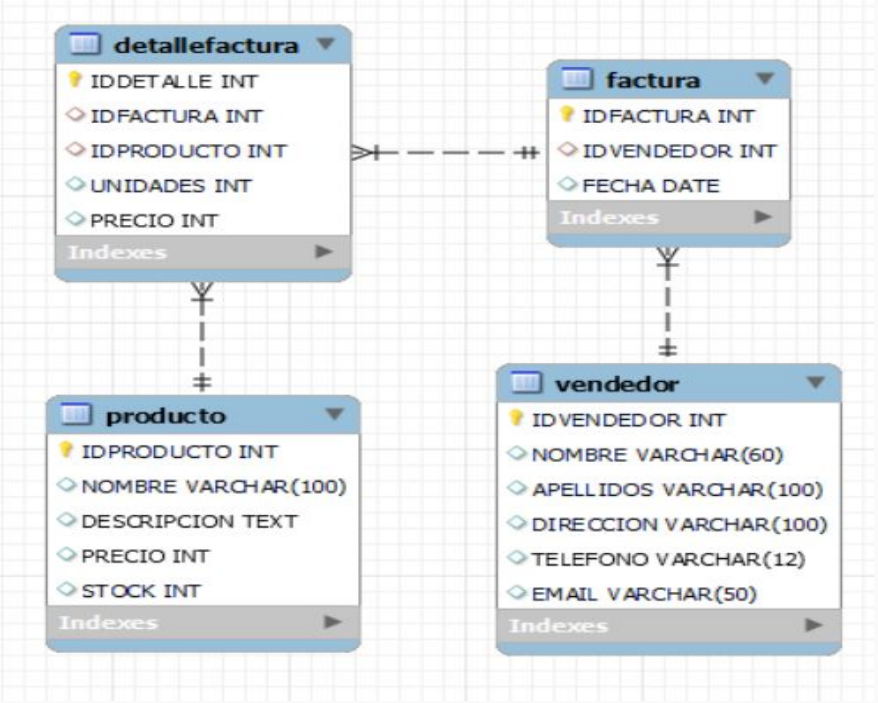
Una vez leído todos los resultados del cursor, procedemos a cerrar y limpiar espacios de memoria con **CLOSE**.

CLOSE cur;

Los cursores mantienen una referencia a los conjuntos de resultados generados por una consulta, y si no se cierran adecuadamente, estos conjuntos permanecerán abiertos en la memoria del servidor, resultando en un consumo innecesario de recursos del servidor, lo que puede llevar a problemas de rendimiento.

CURSOR. Ejemplo

Crea un procedimiento que imprima el código y el total acumulado de ventas del vendedor que más facturó. El procedimiento debe recibir la fecha inicial y la fecha final, para estimar el lapso de tiempo en el que se calculará el acumulado.



CURSOR. Ejemplo

Cosas a tener en cuenta:

- Entradas
  - Las variables fecha_inicio y fecha_final
- Procesos
  - Consultar las facturas que liquidó el vendedor en las fechas estipuladas como entrada.
  - Encontrar los detalles de cada factura asociada al vendedor.
  - Multiplicar el precio por la cantidad de cada detalle y guardarlo en la variable SUMVENTAS.
  - Comparar el SUMVENTAS de cada vendedor mediante un bucle, para obtener el mejor vendedor.
- Salida
  - Mostrar en la pantalla el código y la variable SUMVENTAS del mejor vendedor.

Trigger

Los **triggers** son un tipo de objeto de base de datos que se utilizan para **automatizar ciertas acciones** cuando se realiza **una operación en una tabla**, como **INSERT**, **UPDATE** o **DELETE**.

Los trigger que creamos se quedan **“ligados”** a una tabla en concreto y se encuentran a la espera de la operación que los activa tenga lugar, en el momento que dicho evento ocurre, estos ejecutan el conjunto de instrucciones que tienen programas hacer en su interior. Podremos concretar si actuar **tras realizarse la operación** o actuar **previo a que dicha operación** se lleve a cabo.

Trigger

La sintaxis de los triggers es la siguiente:

CREATE TRIGGER *nombre_trigger*

BEFORE/AFTER *trigger_event*

ON *nombre_tabla* **FOR EACH ROW**

[**FOLLOWS/PRECEDES** *nombre_trigger*]

Bloque_Instrucciones

EVENTOS

Los eventos en MySQL son tareas programadas que se ejecutan automáticamente en un momento específico o de forma recurrente.

Permiten automatizar operaciones como limpieza de datos, backups o actualizaciones periódicas sin intervención manual.



EVENTOS

Lo primero que debemos de hacer es habilitar nuestro servidor para que pueda ejecutar eventos.

```
SET GLOBAL event_scheduler = ON;
```

Tras habilitarlo ya podemos crear nuestros eventos para programar lo que queramos.



event

La sintaxis de los evento es la siguiente:

CREATE EVENT *nombre_evento*

ON SCHEDULE AT/EVERY *momento/frecuencia*

[START ….]

[ON COMPLETION [NOT] PRESERVE]

[ENABLE | DISABLE]

DO